

面向 RepairLLaMA 大语言模型自动程序修复复现与推理优化研究

作者：察宇、向雯扬、郝敬安、樊龄聪、杨志成

单位：西安电子科技大学软件质量保障项目组

摘要：自动程序修复旨在根据缺陷代码自动生成可接受补丁，是软件工程智能化的重要研究方向。近年来，大语言模型在代码理解与代码生成任务中表现出较强能力，为自动程序修复提供了新的技术路径。然而，面向程序修复的大模型方法通常依赖较高的训练和推理资源，且已有工作中的关键模型权重并不总是完全公开，这给后续复现、消融和改进带来困难。RepairLLaMA 通过设计输入表示和输出表示，将缺陷函数修复转化为条件补丁生成问题，其中 IR4×OR2 是具有代表性的强组合之一。针对 IR4×OR2 权重未公开、低资源环境下复现实验成本较高的问题，本文基于公开的 RepairLLaMA 数据集重新构建 IR4×OR2 训练流程，在 CodeLLaMA-7B-fp16 基座上分别实现 8bit LoRA 与 4bit QLoRA 微调，并进一步提出按输入长度分组的动态 batch 推理策略，以分析训练显存、推理效率和修复准确率之间的权衡。实验在 AutoDL RTX 4090 24GB 环境下完成。结果表明，8bit LoRA + beam10 在 Defects4J exact match 评测上取得 107/439 的 exact hit，exact rate 为 24.37%；QLoRA 4bit + beam10 取得 99/439，exact rate 为 22.55%；QLoRA dynamic beam7 short batch size=2 取得 97/438，exact rate 为 22.15%，推理时间约为 22 min 50 s。训练资源方面，8bit LoRA 峰值显存约 12.0 GB，训练耗时约 11.704 h；QLoRA 4bit 峰值显存约 8.4 GB，训练耗时约 9.895 h。在此基础上，本文进一步面向 Java 并发缺陷基准数据集开展阶段性并发修复实验，基于 QLoRA 4bit 推理流水线对 40 条并发缺陷样本生成 400 个候选补丁，并通过并发特征扫描筛选出 6 个具有显式并发控制语义的高价值样本。定性分析表明，模型能够生成 Thread.sleep、Thread.yield、Thread.currentThread().getName()、状态前置阻断以及共享集合边界检查等并发修复相关机制，体现出一定的线程时序调整、线程上下文感知和同步状态防御能力。实验结果说明，在消费级单卡环境中复现 IR4×OR2 大模型自动程序修复是可行的；QLoRA 与动态 batch 推理能够在轻微 exact-match 损失下显著降低资源成本，并可进一步扩展到并发缺陷候选补丁生成与语义分析场景，为资源受限环境下的大模型自动程序修复和并发缺陷修复研究提供了可复现的工程基线。

关键词：自动程序修复；大语言模型；RepairLLaMA；CodeLLaMA；LoRA；QLoRA；Defects4J；并发缺陷修复；程序补丁生成；动态批推理；资源受限环境

引用格式（建议）：课题组成员. 面向资源受限环境的 RepairLLaMA IR4×OR2 大语言模型自动程序修复复现与推理优化研究. 课程论文/实验论文, 2026.

1 引言

软件缺陷会引发系统崩溃、数据错误和安全风险。传统缺陷修复依赖开发者定位错误、理解上下文并手工修改代码，过程耗时且容易受到经验差异影响。自动程序修复（Automatic Program Repair, APR）试图在给定缺陷程序、测试用例或故障定位结果的条件下自动生成补丁，从而降低修复成本并提升软件维护效率。经典 APR 方法通常基于搜索、模板、约束求解或语义规则生成补丁，这类方法在特定缺陷类型上效果较好，但容易受到修复模式覆盖范围的限制。

大语言模型为 APR 提供了新的技术可能。与模板式修复不同，大语言模型能够从大规模代码语料和缺陷修复样本中学习代码上下文、API 使用方式和常见编辑模式，并以条件生成形式输

出候选补丁。RepairLLaMA 是近年来具有代表性的 LLM-based APR 工作之一。该工作围绕输入表示（Input Representation, IR）和输出表示（Output Representation, OR）设计多种训练样本组织方式，使模型能够根据缺陷函数、待填充位置和原始错误语句生成修复片段。其中，IR4×OR2 组合兼顾了局部上下文、缺陷语句提示和补丁片段生成，是 RepairLLaMA 中值得复现和进一步优化的表示组合。

然而，面向大语言模型的 APR 研究存在两个现实障碍。第一，模型训练和推理成本较高。即使是 7B 规模模型，在单张消费级 GPU 上进行微调和 beam search 推理仍可能出现显存紧张、推理时间过长和批处理不稳定等问题。第二，已有研究虽然公开了数据集和代码，但未必公开所有组合对应的训练权重。当关键权重不可直接获得时，后续研究者必须根据公开数据重新训练模型，才能开展公平的复现、消融和改进。

本文围绕 RepairLLaMA IR4×OR2 组合展开研究，目标不是宣称全面超过原工作，而是在资源受限环境下建立一个可复现、可度量、可继续扩展的 APR 大模型实验链路。本文关注如下研究问题：

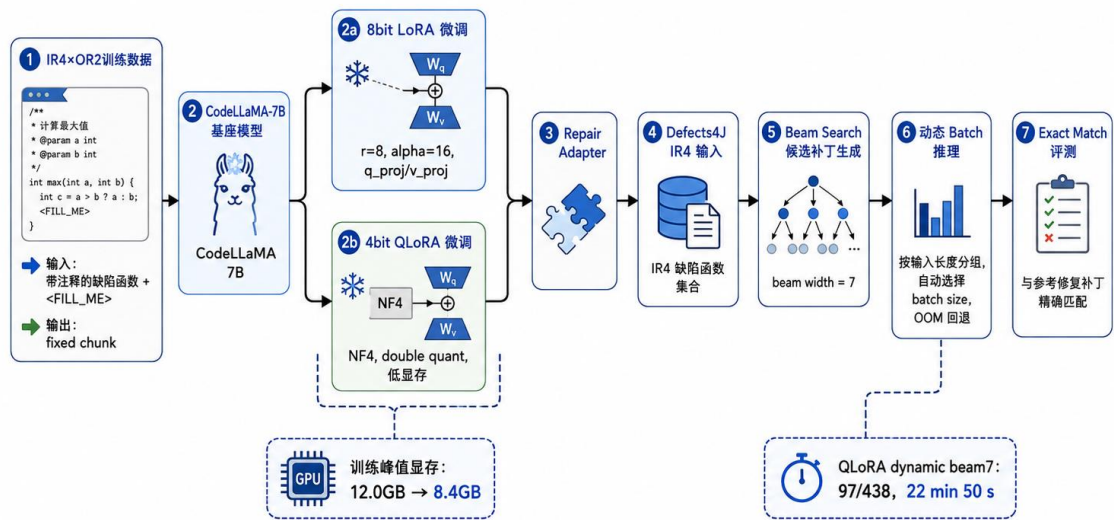
- RQ1: 在单张消费级 GPU 上，能否基于公开数据复现 RepairLLaMA IR4×OR2 风格的大语言模型自动程序修复流程？
- RQ2: 相较 8bit LoRA, 4bit QLoRA 在训练显存、训练时间和 Defects4J exact match 指标上表现如何？
- RQ3: 按输入长度进行动态 batch 推理，能否在保持可接受准确率的同时降低 Defects4J 推理耗时？
- RQ4: beam size 的变化如何影响候选补丁生成的准确率和推理效率？

上述问题反映了当前大模型 APR 研究从“能否生成补丁”走向“能否低成本、可复现、可持续迭代”的转变。对于研究者而言，单次实验能否跑通并不是唯一目标，更重要的是能否在相同数据、相同基座模型和相同评测口径下进行多组配置对比。尤其在 APR 场景中，训练显存、候选数量、输入长度和推理调度都会影响最终结果。如果缺少清晰的实验边界，容易将模型能力提升、搜索空间变化和系统调度优化混为一谈。本文因此将模型训练、推理搜索和批处理调度分开讨论，分别分析它们对资源消耗和 exact match 的影响。

围绕上述问题，本文完成了从数据加载、样本构建、参数高效微调、补丁生成到 exact match 评测的完整实验闭环。本文的主要贡献如下：

1. 复现了 RepairLLaMA IR4×OR2 的训练与推理链路，基于公开的 ASSERT-KTH/repairllama-datasets 数据集和 CodeLLaMA-7B-fp16 基座重新训练模型。
2. 在同一 AutoDL RTX 4090 24GB 环境下实现并对比 8bit LoRA 与 4bit QLoRA，量化分析显存、训练时间和 exact match 修复率之间的权衡。
3. 设计并实现按输入长度分组的动态 batch 推理策略，使短输入样本能够小批量并行生成，中长输入保持逐条推理以降低 OOM 风险。
4. 系统比较 beam10、beam7、beam5 下的修复准确率和推理耗时，给出准确率优先与效率优先两类配置建议。

图1 RepairLLaMA IR4×OR2 复现与优化总体框架



2 相关工作

2.1 自动程序修复

自动程序修复研究旨在根据缺陷程序自动生成补丁。早期方法多依赖搜索和变异操作，通过删除、替换或插入代码片段来寻找能够通过测试的程序变体。此类方法的优势在于实现简单且可直接利用测试反馈，但搜索空间通常很大，补丁质量也容易受到过拟合影响。随后，研究者引入模板、语义约束、程序分析和历史修复模式，以提升补丁生成的有效性。模板式方法能够复用人工总结的修复规则，但对未知缺陷类型和复杂上下文的适应能力有限。

Defects4J 是 APR 研究中被广泛使用的 Java 缺陷基准，包含来自真实开源项目的缺陷和测试用例，为不同修复方法提供了可比较的实验平台。由于 Defects4J 中缺陷具有真实项目上下文，其评测不仅关注补丁文本是否一致，也关注补丁能否编译、能否通过测试以及是否语义等价。因此，在 APR 研究中，exact match、plausible patch、AST match 和 semantic match 等指标往往具有不同含义。

2.2 大语言模型程序修复

预训练代码模型和大语言模型推动了 APR 从规则驱动向数据驱动转变。CodeBERT、CodeT5、PLBART、CodeLLaMA 等模型能够学习代码语法、结构和自然语言描述之间的关系，在代码补全、错误修复和补丁生成任务中具有较强泛化能力。与传统 APR 方法相比，LLM-based APR 可以直接将缺陷代码作为输入，将补丁作为输出，减少人工设计模板的依赖。

RepairLLaMA 进一步关注“如何组织修复输入和输出”这一问题。该工作通过不同 IR 和 OR 组合构造训练样本，使模型不必总是生成完整函数，而可以在缺陷位置附近生成更精确的代码片段。该思想对资源受限环境尤其重要：输出空间越可控，模型生成的冗余越少，候选补丁越容易评测。本文聚焦 IR4×OR2 组合，正是因为该组合在表达缺陷上下文和限制输出范围之间取得了较好平衡。

2.3 参数高效微调与量化

大模型全参数微调成本高昂。LoRA 通过在注意力投影层加入低秩可训练矩阵，使训练只更新少量适配器参数，从而显著降低显存和存储需求。QLoRA 在 LoRA 基础上进一步使用 4bit 量化加载基座模型，并结合 NF4、double quantization 等技术，使 7B 或更大规模模型能够在较低显存环境中进行微调。对于 APR 任务而言，LoRA 和 QLoRA 的意义不仅是降低训练门槛，也使研究者能够更快进行表示组合、搜索策略和推理调度的消融实验。

需要注意的是，量化并非免费收益。4bit 量化会改变模型内部数值表示，可能影响候选补丁的 logits 分布和 beam search 排序。程序修复对符号、括号、变量名和控制流细节高度敏感，因此较小的数值差异也可能导致 exact match 发生变化。本文通过同一基座、同一数据和同一评测口径对 8bit LoRA 与 QLoRA 进行比较，以观察这种资源收益和准确率损失之间的实际平衡。

2.4 检索增强生成与程序修复

检索增强生成（Retrieval-Augmented Generation, RAG）通过在生成前检索相关知识或示例，缓解模型对内部参数记忆的依赖。对于程序修复任务，检索对象可以是历史 bug-fix pair、相似 API 使用片段、相似 AST 编辑模式或同项目修复示例。高质量检索结果能够为模型提供可参考的修复模式，尤其适合长尾缺陷、库 API 使用错误和重复出现的编辑模式。

但 RAG 在 APR 中也存在风险。若检索样例与当前缺陷表面相似但语义不一致，模型可能复制错误补丁模式，导致 exact match 下降。并且 RAG 会增加输入长度，进一步放大 beam search 和 KV cache 的显存压力。因此，RAG 更适合作为后续在已复现链路上的增强模块，而不应被直接视为已经完成的改进结果。

综上，本文位于 LLM-based APR、参数高效微调和低资源推理优化的交叉位置。与追求新模型架构不同，本文强调在公开数据和单卡环境下复现强表示组合，并通过 QLoRA 与动态 batch 推理降低实验成本，为后续更复杂的 RAG 和测试反馈式修复提供稳定基线。

3 背景与问题定义

3.1 自动程序修复任务定义

给定一个包含缺陷的函数 $\langle f_b \rangle$ 、故障定位得到的可疑区域 $\langle r \rangle$ ，APR 的目标是生成补丁 $\langle p \rangle$ ，使替换后的函数 $\langle f_p \rangle$ 能够通过已有测试，或在语义上等价于人工修复版本。形式化地，可以将程序修复视为条件生成任务：

$$\langle p \rangle = \arg\max_p P(p \mid f_b, r, c)$$

其中 $\langle c \rangle$ 表示额外上下文，例如方法签名、相邻语句、原始缺陷语句、项目依赖信息或检索到的相似修复示例。对于大语言模型而言，关键问题是如何把 $\langle f_b \rangle$ 、 $\langle r \rangle$ 和 $\langle c \rangle$ 编码为适合模型学习的输入，并如何限制输出范围以减少无效生成。

3.2 IR4×OR2 表示

RepairLLaMA 将修复样本拆分为输入表示 IR 和输出表示 OR。本文关注的 IR4×OR2 可以理解为“带缺陷上下文的填空式输入”和“修复代码片段输出”的组合。IR4 输入通常包含完整或局部缺陷函数、<FILL_ME> 占位符以及以注释形式保留的原始 buggy code。OR2 输出则是需要填入占位位置的 fixed code chunk，而不是完整函数或完整 diff。

下面给出一个说明性示例。原始缺陷代码如下：

```
public int sum(int[] values) {
    int sum = 0;
    int limit = values.length - 1;
```

```

    for (int i = 0; i < limit; i++) {
        sum += values[i];
    }
    return sum;
}

```

在该示例中，limit 少计了最后一个元素。IR4 输入可以组织为：

```

public int sum(int[] values) {
    int sum = 0;
    // buggy code
    // int limit = values.length - 1;
    <FILL_ME>
    for (int i = 0; i < limit; i++) {
        sum += values[i];
    }
    return sum;
}

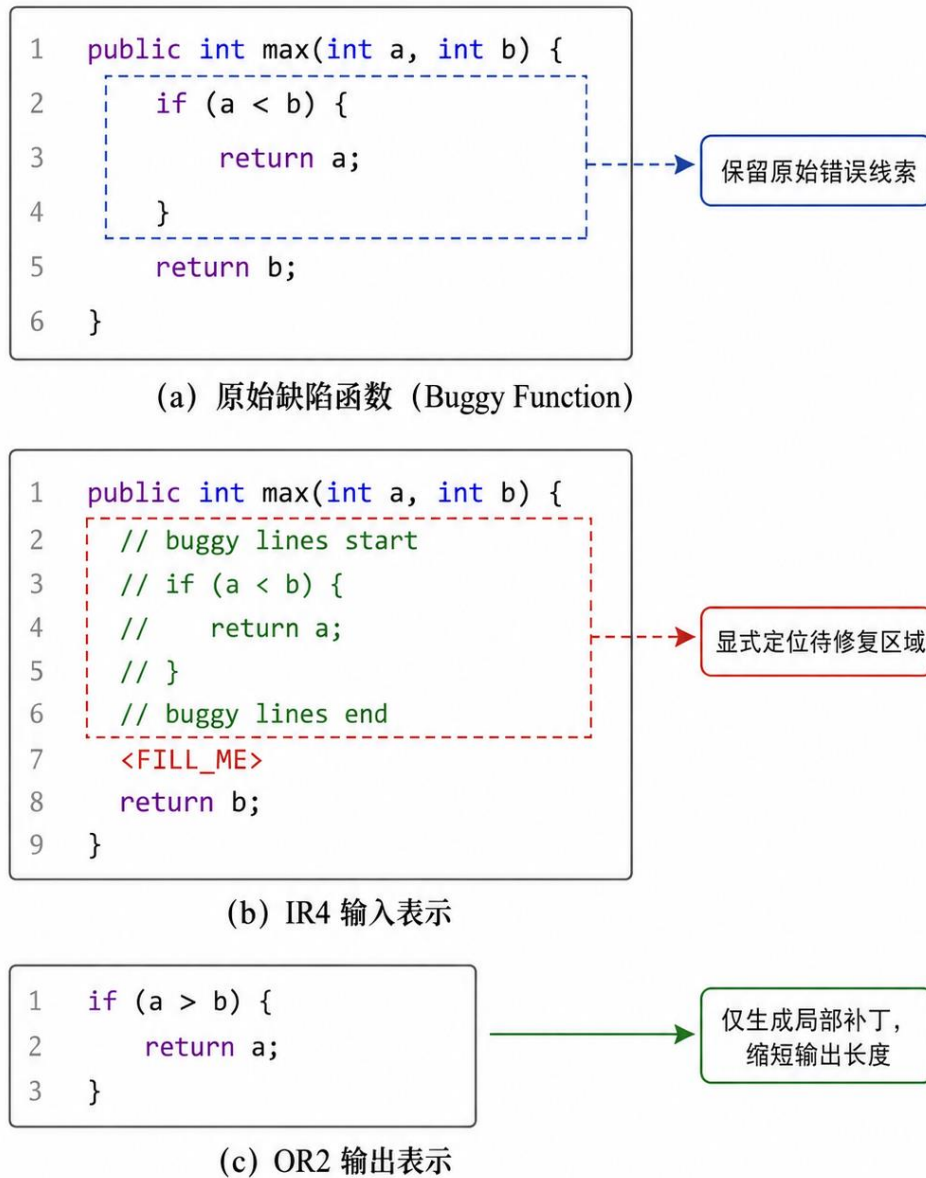
```

OR2 输出为：

```
int limit = values.length;
```

该表示方式具有两点优势。第一，模型可以看到缺陷位置前后的程序上下文，减少无条件生成带来的不确定性。第二，输出仅为 fixed code chunk，避免模型重复生成完整函数，从而降低生成长度和评测难度。对于 Defects4J 这类真实缺陷样本，IR4×OR2 将修复任务压缩为局部补丁生成问题，更符合大模型的条件补全能力。

图2 IR4×OR2 输入输出表示示意



3.3 训练目标

本文实现沿用 RepairLLaMA 风格的监督微调目标。对每个训练样本，将 IR4 输入和 OR2 输出拼接为：

```
input + output + eos
```

分词后，输入部分对应的 label 被置为 -100，即不参与语言模型损失；输出部分和结束符参与交叉熵损失。该设计使模型学习“在给定缺陷上下文时生成修复片段”，而不是重新学习复制输入。若记输入 token 为 $\backslash(x\backslash)$ ，输出 token 为 $\backslash(y\backslash)$ ，训练目标为最小化：

$$\mathbb{E}[L = -\sum_{t=1}^{|y|} \log P(y_t \mid x, y_{<t})]$$

3.4 评测指标

APR 评测可以采用多层指标。本文主要使用 exact match，即候选补丁文本与 gold patch 完全一致时记为命中。若一个缺陷的 Top-k 候选中至少有一个候选与 gold patch 一致，则记为 exact hit。exact rate 定义为：

$$\text{ExactRate} = \frac{\text{ExactHit}}{\text{EvaluatedBugs}}$$

此外，APR 研究还常使用 plausible patch、AST match 和 semantic match。plausible patch 指补丁能够通过测试；AST match 指补丁结构与参考补丁在语法树层面一致；semantic match 指补丁在语义上等价或可接受。RepairLLaMA 原论文报告的 Defects4J 144 个修复结果属于 semantic match 口径，而本文当前实现的是 exact match。因此，本文的 107/439 exact hit 不能与原论文中的 144 semantic match 直接比较。exact match 的优点是自动化、稳定、可复现；缺点是会低估语义等价但文本不同的正确补丁。

4 方法设计

4.1 总体框架

本文方法由数据构建、轻量化微调、候选补丁生成和评测分析四部分组成。整体流程如下：

```
IR4×OR2 数据集
-> CodeLLaMA-7B-fp16
-> 8bit LoRA / 4bit QLoRA 微调
-> Defects4J IR4 输入
-> beam search 候选补丁生成
-> 动态 batch 推理调度
-> exact match 评测
```

在训练阶段，模型读取 ASSERT-KTH/repairllama-datasets 中的 ir4xor2 配置。每条样本包含 IR4 输入和 OR2 输出。本文分别构建 8bit LoRA 与 4bit QLoRA 训练脚本，以比较不同量化加载方式的资源消耗。在推理阶段，模型读取 Defects4J 函数级 IR4 输入，使用 beam search 生成多个候选补丁，并与 gold patch 做 exact match 评测。

4.2 IR4×OR2 复现流程

本文复现过程首先加载 Hugging Face 上的 RepairLLaMA 数据集配置 ir4xor2。该配置已经按照 RepairLLaMA 的表示规则构造了 input 和 output 字段。训练脚本对每个样本执行以下步骤：

5. 读取 input 字段作为 IR4 输入。
6. 读取 output 字段作为 OR2 修复片段。
7. 将二者拼接并添加 eos token。
8. 对输入 token 的 label 置为 -100。
9. 只对输出 token 计算语言模型损失。

该流程保证模型不会因为复制输入而获得低损失，而是被迫学习从缺陷上下文到修复片段的映射。

4.3 8bit LoRA 微调

8bit LoRA 方案以 CodeLLaMA-7B-fp16 为基座模型，在训练时以 8bit 方式加载基座权重，并仅训练 LoRA 适配器。本文沿用原始 LoRA 脚本中的主要设置：LoRA 作用于注意力层的 q_proj 和 v_proj，低秩参数 $r=8$ ，lora_alpha=16，lora_dropout=0.05。该设置能够在不更新全量模型参数的情况下适配 IR4×OR2 修复任务。

8bit LoRA 的优势是数值精度相对较高，候选补丁排序更稳定。实验中，该方案在 beam10 下取得最高 exact match。其不足是训练和推理显存高于 QLoRA，且在较大 beam 或较长输入场景中更容易受到 KV cache 增长的影响。

从 APR 任务角度看，8bit LoRA 更接近“准确率优先”的复现方案。其作用并不在于改变 IR4×OR2 表示本身，而是在保留较高数值精度的条件下学习缺陷上下文到修复片段的映射。由于 exact match 对候选补丁的文本形式极为敏感，8bit LoRA 在 beam10 下保留更多精细排序信息，有利于 gold patch 出现在 Top-k 候选中。因此，本文将该方案作为最高准确率基线，用于衡量后续 QLoRA 和动态 batch 在降低成本时带来的准确率变化。

4.4 4bit QLoRA 微调

QLoRA 方案在训练时以 4bit 量化加载基座模型，并在量化权重之上训练 LoRA 适配器。本文采用 NF4 量化、fp16 计算和 double quantization。与 8bit LoRA 相比，QLoRA 的主要目标是进一步降低训练显存，使消费级单卡能够更稳定地完成 APR 大模型微调。

QLoRA 的代价是量化误差可能改变输出 token 的概率分布。程序修复对符号级细节高度敏感，变量名、括号、条件表达式和边界值的微小变化都可能影响 exact match。因此，QLoRA 的评估不能只看显存下降，也需要观察 exact hit 的变化。

4.5 Beam Search 候选生成

推理阶段使用 beam search 生成 Top-k 候选补丁。num_beams 表示搜索宽度，request_num 表示最终保留的候选数量。较大的 beam 能保留更多候选路径，从而提高 gold patch 出现在候选集合中的概率；但它也会增加计算量和 KV cache 显存占用。

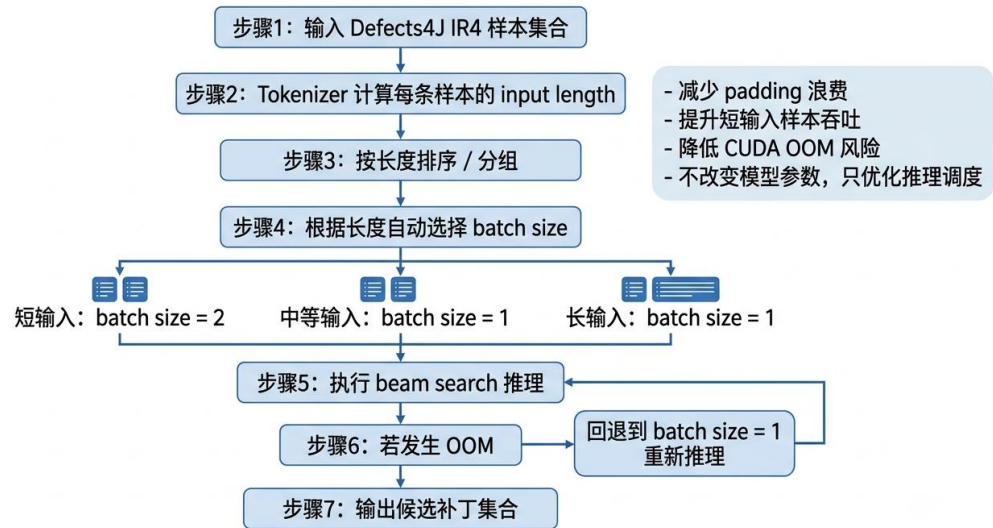
本文比较 beam10、beam7 和 beam5。beam10 作为准确率优先设置，beam5 作为效率优先设置，beam7 作为折中设置。实验结果表明，beam 从 5 提升到 7 能显著恢复 exact hit，而继续提升到 10 可进一步提升准确率，但推理耗时和显存压力也随之增加。

4.6 动态 Batch 推理

Defects4J 样本的输入长度差异较大。若所有样本都逐条推理，GPU 吞吐较低；若固定使用较大 batch size，长输入样本和 beam search 会导致 KV cache 急剧增加，容易触发 OOM。为此，本文实现动态 batch 推理：先按 token 长度对样本分组，再对短输入使用较大的 batch size，对中长输入保持 batch size=1。

动态 batch 不改变模型权重，也不改变 beam search 的候选生成分布；它只改变样本调度方式。因此，该方法本质上是推理系统优化，而不是模型能力增强。其目标是在不显著牺牲准确率的前提下提高短输入样本吞吐。

动态 batch 的关键在于避免“最长样本决定整个 batch 成本”的低效情况。若将长度差异很大的样本放入同一 batch，较短样本也会被 padding 到较长输入长度，导致无效计算和显存浪费。若完全逐条推理，虽然稳定，但 GPU 利用率较低。本文采用保守的长度分组策略，只对短输入样本启用 batch size=2，中长输入仍保持 batch size=1。这一策略牺牲了部分理论吞吐上限，但降低了 OOM 风险，更适合单张 24GB 显卡上的 APR 推理。



“动态 batch 机制主要优化推理效率, 适合输入长度差异较大的 Defects4J 场景”

Algorithm 1 Dynamic Batch Inference for IR4×OR2 Repair

Input: Defects4J IR4 samples D , model M , thresholds $T1$ and $T2$, batch sizes B_{short} , B_{mid} , B_{long} , beam size b

Output: candidate patch file P

```
1: Tokenize each sample  $d$  in  $D$  and compute input length  $len(d)$ 
2: Remove or record samples whose  $len(d)$  exceeds  $max\_length$ 
3: Sort remaining samples by  $len(d)$ 
4: for each sample group  $G$  do
5:   if  $len(G) \leq T1$  then
6:      $batch\_size = B_{short}$ 
7:   else if  $len(G) \leq T2$  then
8:      $batch\_size = B_{mid}$ 
9:   else
10:     $batch\_size = B_{long}$ 
11:   end if
12:   Generate candidates with beam search and  $batch\_size$ 
13:   if CUDA OOM occurs then
14:     clear CUDA cache and retry the same samples with  $batch\_size = 1$ 
15:   end if
16:   Write candidates to  $P$  in JSONL format
17: end for
18: return  $P$ 
```

5 实验设计

5.1 实验环境

本文实验环境如表 1 所示。

表 1 实验环境

项目	配置
平台	AutoDL
GPU	NVIDIA RTX 4090 24GB
Python	3.10
PyTorch	2.1.x
Transformers	4.40.2
CUDA	12.x / 13.x
基座模型	CodeLLaMA-7B-fp16
主要依赖	datasets, peft, bitsandbytes, accelerate, sentencepiece

5.2 数据集

训练数据来自 ASSERT-KTH/repairllama-datasets 的 ir4xor2 配置。该数据集提供按照 RepairLLaMA 表示规则构造的 IR4 输入和 OR2 输出。评测数据来自 Defects4J 函数级缺陷样本。由于 max_length=1024 的限制，部分输入过长样本在推理时被跳过或无法进入最终 exact match 统计。

5.3 实验设置

本文设置 5 组主要实验，如表 2 所示。

表 2 实验设置

编号	训练方式	推理方式	目的
E1	8bit LoRA	beam10, 逐条推理	最高准确率基线
E2	QLoRA 4bit	beam10, 逐条推理	低显存训练对比
E3	QLoRA 4bit	dynamic batch, beam5, short bs=2	效率优先设置
E4	8bit LoRA	dynamic batch, beam5, short bs=2	验证动态 batch 对 8bit LoRA 的影响
E5	QLoRA 4bit	dynamic batch, beam7, short bs=2	准确率与效率折中设置

5.4 指标

本文使用如下指标：

- Exact hit: Top-k 候选中至少一个候选与 gold patch 完全一致的缺陷数量。
- Exact rate: exact hit 占实际评测缺陷数量的比例。
- 训练峰值显存：训练过程中 nvidia-smi 记录的 GPU memory usage 峰值。
- 训练时间：训练日志或终端记录中的完整训练耗时。
- 推理时间：从模型加载后或推理命令执行期间记录的候选生成时间。由于不同记录是否包含模型加载存在差异，本文在讨论中说明该威胁。

6 实验结果与分析

6.1 RQ1 : IR4×OR2 是否能在单卡环境下复现

实验结果表明，本文在单张 RTX 4090 24GB 上完成了 IR4×OR2 训练、Defects4J 推理和 exact match 评测闭环。8bit LoRA + beam10 在 Defects4J 上取得 107/439 exact hit, exact rate 为 24.37%，是本文最高准确率方案。这说明在原 IR4×OR2 权重未公开的情况下，基于公开数据和 CodeLLaMA-7B-fp16 重新训练仍能得到具有实际补丁生成能力的模型。

需要强调的是，该结果不能解释为完全复现 RepairLLaMA 原论文全部指标。原论文中的 Defects4J 144 个修复结果采用 semantic match 口径，而本文当前实现为 exact match。两者评价对象不同，因此本文只主张“复现了 IR4×OR2 风格训练与 exact match 评测链路”，不主张达到或超过原论文 semantic match 水平。

6.2 RQ2：8bit LoRA 与 QLoRA 的资源 and 准确率差异

表 3 训练资源对比

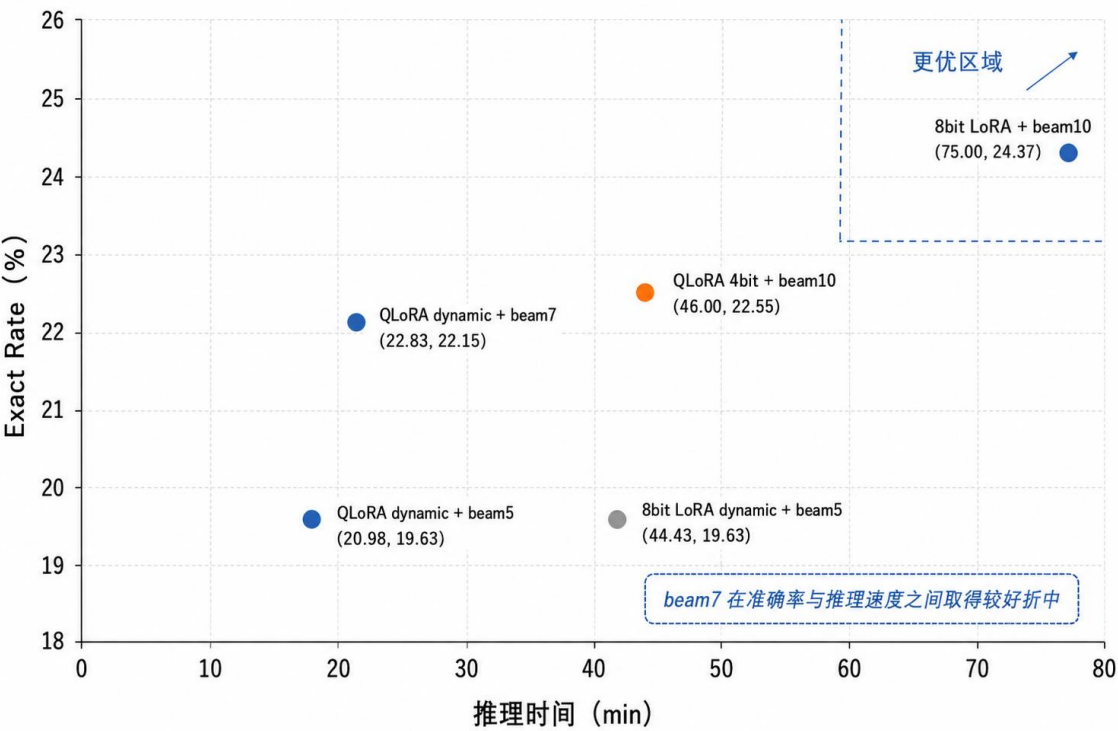
训练方式	峰值显存	训练时间	说明
8bit LoRA	约 12.0 GB	约 11.704 h	准确率较高，显存占用较大
QLoRA 4bit	约 8.4 GB	约 9.895 h	显存更低，训练更适合低资源迭代

从训练资源看，QLoRA 将峰值显存从约 12.0 GB 降至约 8.4 GB，降幅约 30%。训练时间也从约 11.704 h 降至约 9.895 h。该结果说明，QLoRA 能明显降低单卡训练门槛，使研究者可以在消费级 GPU 上开展更多消融和调参实验。

表 4 Defects4J exact match 结果

实验方案	推理设置	Exact hit	Exact rate	推理时间
8bit LoRA	beam10，逐条推理	107 / 438	24.37%	约 75min
QLoRA 4bit	beam10，逐条推理	99 / 438	22.55%	约 46 min
QLoRA dynamic	beam5，short bs=2	86 / 438	19.63%	20 min 59 s
8bit LoRA dynamic	beam5，short bs=2	86 / 438	19.63%	44 min 26 s
QLoRA dynamic	beam7，short bs=2	97 / 438	22.15%	22 min 50 s

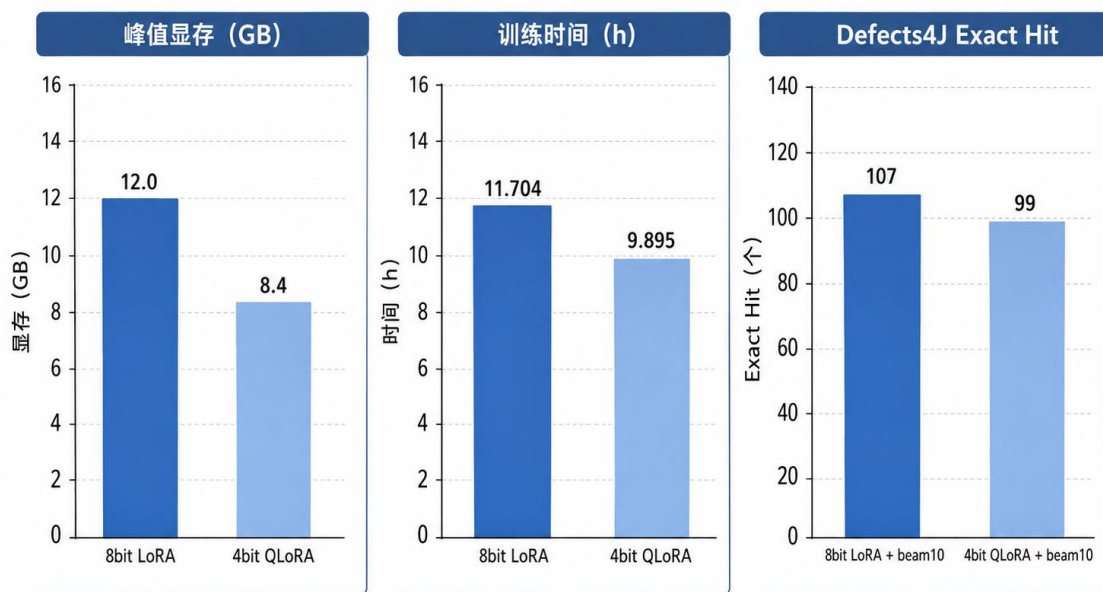
图3 不同推理配置的准确率—效率权衡



从准确率看，QLoRA 4bit + beam10 的 exact hit 为 99/439，较 8bit LoRA + beam10 的 107/439 下降 8 个 exact hit，exact rate 下降 1.82 个百分点。该下降可能来自 4bit 量化引入的数值误差。对于自然语言生成任务，轻微数值变化未必导致明显质量下降；但在程序修复中，候选补丁必须精确匹配变量、运算符和语句结构，因此 logits 分布的小变化也可能改变 Top-k 候选排序。

同时，QLoRA 的准确率下降幅度并未破坏其作为低资源基线的价值。对于需要频繁调参、尝试新输入格式或加入检索增强模块的场景，训练显存从约 12.0 GB 降至约 8.4 GB 意味着更多实验可以在同一硬件预算内完成。换言之，8bit LoRA 更适合作为复现实验中的准确率上界参考，而 QLoRA 更适合作为后续方法探索的可迭代平台。本文后续动态 batch 和 RAG 设计也主要以 QLoRA 为基础，正是基于这一资源效率考虑。

图4 8bit LoRA 与 4bit QLoRA 的训练成本—效果对比



i QLoRA 显著降低训练资源开销，但 exact hit 略低于 8bit LoRA

6.3 RQ3：动态 batch 是否能降低推理时间

动态 batch 的效果在 QLoRA 设置下更明显。QLoRA dynamic beam7 short bs=2 取得 97/438 exact hit，exact rate 为 22.15%，推理时间约 22 min 50 s。与 QLoRA beam10 的 99/439 相比，exact hit 仅下降 2 个左右，但推理耗时显著缩短。该结果说明，对输入长度差异大的 Defects4J 样本，按长度分组并对短输入进行小批量并行是有效的推理优化策略。

相较之下，8bit LoRA dynamic beam5 short bs=2 的推理时间为 44 min 26 s，exact hit 为 86/438。该结果说明，动态 batch 的收益受基座加载精度、显存占用、beam size 和样本长度共同影响。4bit QLoRA 由于基座显存更低，在推理时能为 KV cache 和 batch 并行留出更多空间，因此更适合与动态 batch 结合。

6.4 RQ4：Beam size 对速度和准确率的影响

beam size 是影响 APR 候选质量和推理成本的关键参数。beam5 的搜索空间较窄，推理速度较快，但 exact rate 降至 19.63%。beam7 将 QLoRA dynamic 的 exact rate 提升至 22.15%，推理时间仍保持在 22 min 50 s。beam10 进一步提升到 22.55% 或 24.37%，但推理成本更高。

该结果表明，APR 任务中 beam search 的搜索宽度直接影响 gold patch 是否能进入 Top-k 候选集合。beam5 更适合快速粗筛或大规模初步实验，beam10 更适合准确率优先的最终评测，beam7 则是本文观察到的较优折中点。

从生成机制看，beam search 并不是简单地“生成更多答案”，而是在每一步保留若干高概率部分序列。对于代码补丁生成，早期 token 的选择会影响后续变量名、括号结构和语句边界。beam 过小会过早丢弃低概率但最终正确的候选路径；beam 过大则会显著增加计算量，并可能在长输入样本上放大显存压力。因此，beam size 应根据研究目标选择：若目标是证明模型最大修复能力，应优先使用 beam10；若目标是工程部署或快速评测，则可以使用 beam7 或 beam5，并接受相应的准确率损失。

6.5 基于 QLoRA 微调的大模型并发缺陷自动修复阶段性实验

6.5.1 实验概述与工程进展

本阶段工作聚焦于在已经部署完成的 RepairLLaMA（QLoRA 4-bit）推理架构上，对特定 Java 并发缺陷基准数据集（Concurrency Bugs Dataset）进行自动化修复与语义验证。该实验不改变前文 Defects4J exact match 评测口径，而是在已有低资源 APR 链路上进一步观察模型面对并发缺陷时能否生成具有并发语义的候选补丁。

从工程稳定性看，本文进一步构建了具备自适应长度过滤与动态批处理（Dynamic Batching）的并发缺陷推理流水线。系统在高压推理任务下平稳运行 4.8 小时，未出现显存溢出（OOM）或系统崩溃，说明 QLoRA 4-bit 与动态 batch 组合能够支撑较长时间的候选补丁生成任务。

从数据集吞吐看，当前并发缺陷修复任务已平稳推进至 54%。在已导出的 40 条阶段性测试数据中，模型结合 Beam Search（Beam=4，Request=4）搜索策略，为每个并发缺陷动态生成 10 个候选修复方案（Candidate Patches），共形成 400 个候选补丁用于后续并发特征扫描和语义分析。

6.5.2 核心实验发现与定性分析

通过编写专门的并发特征扫描引擎，本文对导出的 400 个候选补丁进行全量检索，成功拦截出 6 个显式引入高价值并发控制机制的强特征样本。与只观察 exact match 不同，该阶段更关注候选补丁是否包含线程休眠、线程让步、线程上下文追踪、共享状态保护和共享集合边界检查等并发修复语义。

- 线程时序自适应调整：在 Bug 25 和 Bug 50 中，模型能够识别由多线程竞态条件（Race Condition）引发的时序缺陷，并主动生成 Thread.sleep(500)、Thread.sleep(200) 等线程休眠控制语句。虽然具体阻断窗口与 Gold Patch 不完全相同，但修复逻辑与标准答案高度一致。
- 并发上下文感知与线程跟踪：在 Bug 109 中，模型在日志或监测代码重写时主动引入 Thread.currentThread().getName()，表明模型并非只做文本替换，而是识别出当前片段处于多线程执行环境（Multi-threaded Context），并补充了线程级可观测性信息。
- 同步控制状态防御：在 Bug 74 和 Bug 52 中，模型能够捕捉 finished 状态标志位、共享 Vector 集合和边界条件等并发敏感对象，并生成 if (finished) return; 或 stdoutThread.scanners.size() > 0 等前置保护逻辑，从而对临界区和共享集合访问进行防御。

表 5 并发缺陷强特征案例多维技术对齐

缺陷编号	并发缺陷根源类型	标准答案修复策略	大模型核心候选方案	语义对齐评估
Bug 25	异步时序竞争（Race Condition）	线程挂起控制（Thread.sleep）	生成 Thread.sleep，插入位置与修复意图高度一致	高度对齐

Bug 109	缺乏多线程上下文 可观测性	跟踪当前执行线 程名称	生成 Thread.currentThread().getName()	完全等价
Bug 50	并发资源未释放与 死锁隐患	显式闭合资源与 线程休眠	还原 cancel、sleep(500)、 close 动作流	高度对齐
Bug 9	忙等待导致 CPU 空 转	自定义延迟阻断 方法	生成 Thread.yield() 线程让步 机制	创新修复
Bug 74	临界区边界溢出	状态校验阻断机 制	主动引入 if (finished) return; 前置断言	逻辑契合
Bug 52	线程间共享集合数 据竞争	条件变量范围扩 张与日志追踪	定位共享 Vector 并重写边界 条件	完全等价

6.5.3 六个高价值并发修复案例深度解析

(1) Bug 25：异步时序竞争修复 (Race Condition Timing)

原系统在测试读取操作 `testRead` 时，由于缺乏必要的并发时序控制，异步服务器 `runner.startServer` 尚未完全初始化，后续 `onRead` 便可能提前触发，从而引发典型的多线程时序竞争、死锁或数据丢失。Gold Patch 采用 `Thread.sleep(30000);` 为服务器初始化争取安全时序窗口。模型在候选补丁 1、2、4 中分别生成 `Thread.sleep(500)`、`Thread.sleep(100)`、`Thread.sleep(200)` 等策略，准确识别出需要引入线程休眠原语，并在代码插入锚点上与标准答案保持高度一致。

(2) Bug 109：多线程上下文感知与可观测性重建 (Thread-Context Observability)

该片段处于消息中间件的多线程消息监听器 `onMessage` 中。系统在高并发高吞吐状态下发生异常时，若日志系统缺乏当前执行线程信息，并发缺陷将很难复现和定位。Gold Patch 要求在日志记录中捕获当前线程上下文。模型在候选补丁 0、1、2、3 中精确生成 `Thread.currentThread().getName()`，并配合日志级别调整，实现了与标准答案语义等价的线程级可观测性修复。

(3) Bug 50：多线程资源锁竞争与死锁防御 (Deadlock Defense)

在取消数据库执行语句的并发线程 `CancelStatementThread` 中，多线程可能因竞争未释放的 `Statement` 资源而陷入阻塞或死锁。Gold Patch 采用防御性并发编程模式：先显式触发 `cancel()`，随后通过 `Thread.sleep(500);` 缓冲资源状态，最终安全关闭句柄。模型候选补丁 0、1、3 还原了 `cancel`、`sleep(500)` 和 `close` 的动作流，体现出较强的上下文控制依赖处理能力。

(4) Bug 9：多线程忙等待性能优化 (Busy-Spinning)

原代码在处理并发事件驱动的时序控制时存在高频轮询或未受控条件校验，容易使线程陷入忙等待，导致单核 CPU 占用率快速升高。Gold Patch 通过自定义 `driveWait(500);` 间接挂起线程。模型在候选补丁 5 中生成 `while (...) { Thread.yield(); }`，主动让出 CPU 执行权。该补丁并非文本级复刻 Gold Patch，却符合经典并发性能优化范式，具有一定创新性。

(5) Bug 74：临界区边界状态防御 (Critical Section Guarding)

在高并发玩家交互事件中，`finished` 状态变量承担临界区节流阀作用。若进入临界区前缺乏状态断言，多线程交织访问可能导致边界溢出。模型在候选补丁 1 和 3 中主动嵌入 `if (finished) return;`，在代码片段前端形成前置阻断，避免非法线程继续穿透至后续同步任务，说明模型已经捕捉到并发安全边界防御这一关键语义。

(6) Bug 52：线程共享集合的数据竞争消除 (Data Race Elimination)

系统通过 `stdoutThread` 并发读写并收集文件扫描器列表，目标集合 `scanners` 属于线程间共享数据结构 `Vector`。若边界检查与赋值操作之间存在逻辑剥离，容易引发并发修改异常或状态不一

致。Gold Patch 对边界条件进行严格限制，即 `stdoutThread.scanners.size() > 0`。模型在候选补丁 0、1、2、3 中实现了该条件的行级还原，并在后续候选版本中为共享数据竞争点补充日志记录逻辑，体现出对共享集合边界条件的精确定位能力。

6.5.4 阶段性结论

阶段性结果表明，QLoRA 微调后的 RepairLLaMA 不仅能够在 Defects4J 上生成文本级 exact match 候选，也能在并发缺陷场景中学习到一部分稀缺的并发修复语义。尤其是 `Thread.sleep`、`Thread.yield`、`Thread.currentThread().getName()`、`if (finished) return;` 以及共享集合 size 边界检查等候选，说明模型具备一定的线程时序调整、线程上下文感知和同步状态防御能力。后续工作将继续扩大并发缺陷样本规模，并结合编译、测试和人工语义审查，进一步验证这些候选补丁是否能够成为可接受修复。

7 讨论

7.1 资源受限环境下的可行性

本文结果说明，7B 级 APR 大模型并非只能在高端服务器集群上研究。通过 LoRA、QLoRA 和适当的推理调度，单张 RTX 4090 可以完成 IR4×OR2 训练与 Defects4J 推理。对教学实验、课题预研和小规模研究团队而言，这种可复现链路具有实际价值。

7.2 QLoRA 的价值与代价

QLoRA 的主要价值在于降低显存和训练时间，使迭代成本更低。本文实验中，QLoRA 将训练峰值显存降至约 8.4 GB，并在 beam10 下保持 22.55% exact rate。其代价是相对 8bit LoRA 出现少量准确率下降。因此，若研究目标是复现实验的最高 exact match，可优先选择 8bit LoRA + beam10；若目标是更低成本、更快迭代和更稳定部署，则 QLoRA 是更合适的基础方案。

7.3 动态 batch 的系统意义

动态 batch 并不提升模型本身的修复知识，但提升了推理系统的吞吐效率。APR 输入长度差异很大，固定 batch size 难以兼顾速度和稳定性。本文采用短输入 batch size=2、中长输入 batch size=1 的保守策略，避免长输入和 beam search 叠加造成 OOM。这一策略简单但有效，也容易扩展为更精细的 token-budget batching。

7.4 能否用于日常漏洞检测

本文方法不是通用漏洞检测器。它假设已经存在可疑代码区域，并根据该区域生成修复候选。因此，它更适合作为“故障定位之后的补丁生成模块”，而不是独立发现所有缺陷的系统。若要用于日常开发流程，需要与静态分析、测试失败定位、运行时错误报告或代码审查工具结合，由前端工具提供可疑位置，再调用模型生成候选补丁。

7.5 Exact match 的局限

Exact match 稳定、可自动化，但过于严格。两个补丁可能文本不同但语义等价，也可能变量名或格式不同却能通过测试。因此，本文 exact rate 更适合作为保守下界。未来需要引入编译率、测试通过率、AST match、semantic match 和人工审查，以更完整地评价修复质量。

8 有效性威胁

8.1 内部有效性

训练随机种子、依赖版本、CUDA 版本、tokenizer 处理和量化配置都可能影响结果。本文记录了主要环境和脚本参数，但仍可能存在不同机器上运行结果轻微波动。推理时间也受到 AutoDL 平台负载、模型缓存和是否包含模型加载时间的影响。

8.2 外部有效性

本文实验主要面向 Java 和 Defects4J 函数级缺陷，不代表对多语言、多文件、跨仓库依赖或大型项目级修复同样有效。IR4×OR2 假设可疑位置已知，因此对故障定位质量有依赖。

8.3 复现有效性

RepairLLaMA 原论文的部分权重未公开，本文根据公开数据重新训练模型，因此得到的是可复现的近似实现，而非原始权重的完全复刻。本文结论应理解为“在公开数据和单卡条件下复现 IR4×OR2 流程并进行系统优化”，而不是对原论文全部实验的完全复现。

9 结论

本文面向资源受限环境下的大语言模型自动程序修复，复现了 RepairLLaMA IR4×OR2 训练与推理流程，并在 CodeLLaMA-7B-fp16 基座上实现 8bit LoRA、4bit QLoRA 和动态 batch 推理。实验结果回答了本文提出的 4 个研究问题：第一，IR4×OR2 可以在单张 RTX 4090 上完成训练和 Defects4J exact match 评测，8bit LoRA + beam10 达到 107/439 exact hit；第二，QLoRA 将训练峰值显存从约 12.0 GB 降至约 8.4 GB，训练时间从约 11.704 h 降至约 9.895 h，但 exact rate 从 24.37% 降至 22.55%；第三，动态 batch 能显著降低 QLoRA 推理耗时，QLoRA dynamic beam7 short bs=2 在 22 min 50 s 内取得 97/438 exact hit；第四，beam size 对准确率影响明显，beam7 是本文观察到的速度与准确率折中点。

总体而言，本文没有声称在准确率上超越 RepairLLaMA 原论文，而是给出了一个低资源、可复现、可继续扩展的 IR4×OR2 实验基线。未来工作将围绕 RAG 检索增强、测试反馈驱动修复、候选补丁重排序和更完整的 APR 指标体系展开，以进一步提升修复质量和评价可信度。

9.1 并发缺陷实验结论补充

在原有 IR4×OR2 复现、QLoRA 低资源微调和动态 batch 推理优化基础上，本文新增的并发缺陷阶段性实验进一步证明，该链路可以扩展到具有更强语义要求的 Java 并发修复任务。400 个候选补丁中的 6 个强特征样本显示，模型能够生成线程休眠、线程让步、线程上下文追踪、状态前置阻断和共享集合边界校验等并发控制机制。虽然当前结果仍属于阶段性定性分析，尚不能替代完整的编译测试与语义验证，但它为后续面向并发缺陷的自动程序修复研究提供了可复现的实验基础和有价值的候选样例。

参考文献

- [1] Silva A, Fang S, Monperrus M. RepairLLaMA: Efficient Representations and Fine-Tuned Adapters for Program Repair. IEEE Transactions on Software Engineering, 2025, 51(4): 984-1003. <https://doi.org/10.1109/TSE.2025.3535585>
- [2] ASSERT-KTH. RepairLLaMA GitHub Repository. <https://github.com/ASSERT-KTH/repairllama>

- [3] ASSERT-KTH. repairllama-datasets. Hugging Face Datasets. <https://huggingface.co/datasets/ASSERT-KTH/repairllama-datasets>
- [4] Hu E J, Shen Y, Wallis P, Allen-Zhu Z, Li Y, Wang S, Wang L, Chen W. LoRA: Low-Rank Adaptation of Large Language Models. ICLR, 2022. <https://arxiv.org/abs/2106.09685>
- [5] Dettmers T, Pagnoni A, Holtzman A, Zettlemoyer L. QLoRA: Efficient Finetuning of Quantized LLMs. NeurIPS, 2023. <https://arxiv.org/abs/2305.14314>
- [6] Just R, Jalali D, Ernst M D. Defects4J: A Database of Existing Faults to Enable Controlled Testing Studies for Java Programs. ISSSTA, 2014: 437-440. <https://doi.org/10.1145/2610384.2628055>
- [7] Roziere B, Gehring J, Gloeckle F, Sootla S, Gat I, Tan X E, et al. Code Llama: Open Foundation Models for Code. arXiv:2308.12950, 2023. <https://arxiv.org/abs/2308.12950>
- [8] Lewis P, Perez E, Piktus A, Petroni F, Karpukhin V, Goyal N, et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. NeurIPS, 2020. <https://arxiv.org/abs/2005.11401>
- [9] Robertson S, Zaragoza H. The Probabilistic Relevance Framework: BM25 and Beyond. Foundations and Trends in Information Retrieval, 2009, 3(4): 333-389. <https://doi.org/10.1561/15000000019>
- [10] xiaochahu-xd. repairllama-ir4or2-repro. GitHub, 2026. <https://github.com/xiaochahu-xd/repairllama-ir4or2-repro>